```
    public function getFighters(): array
    {
        return $this->fighters;
    }
}
```

The `::addMember()` method is where we inspect the incoming fighters or healers and place them in the correct private array as they join. Each `$member` object is used to create a new `ReflectionObject`, and from there we can access any attributes via `ReflectionObject::getAttributes()`, which will return an array of `ReflectionAttribute` objects (one per attribute). All we need to do is loop over each object's attributes looking for a match between the attribute class name and `ReflectionAttribute::getName()` (which will be the fully qualified name including any namespaces). By using attributes, we have kept our interfaces free of extra properties (or methods) for figuring out if they fight or heal. Since we've gone to all this trouble, let's make use of our attributes and classes:

```
$friartuck = new Monk();
$conan = new Barbarian();
$galadriel = new Cleric();

$genremuddle = new Party();

$genremuddle->addMember($friartuck);
$genremuddle->addMember($conan);
$genremuddle->addMember($galadriel);

var_dump($genremuddle->getHealers());
var_dump($genremuddle->getFighters());

Output:
array(2) {
  [0]=>
  object(Monk)#1 (0) {
  }
  [1]=>
  object(Cleric)#3 (0) {
  }
}
```